

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 4 Exam Notes: Constructors and Destructors

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Fundamentals of Constructors	2
1.1	Rules and Characteristics	2
1.2	Types of Constructors	2
2	Deep Copying and Shallow Copying	2
2.1	Shallow Copy vs. Deep Copy	2
3	Constructor Overloading & Destructors	3
3.1	Constructor Overloading and Default Arguments	4
3.2	Destructors	4

1. Fundamentals of Constructors

Constructors are special member functions of a class designed specifically to initialize objects during instantiation.

1.1 Rules and Characteristics

A constructor must follow strict compiler rules:

- **Naming:** It must have the exact same name as the class.
- **Return Type:** It does not have any return type, not even `void`.
- **Execution:** It is automatically invoked by the system when an object of the class is created.
- **Implicit/Explicit Invocation:**

Implicit call: `Complex c1(5, 10);` (1)

Explicit call: `Complex c1 = Complex(5, 10);` (2)

- **Restrictions:** It cannot be declared `virtual`, `static`, or `const`.

1.2 Types of Constructors

1. **Default Constructor:** A constructor that accepts no arguments. If no constructor is written, the compiler automatically generates a default constructor.
2. **Parameterized Constructor:** A constructor that accepts arguments, allowing objects to be initialized with custom values at creation time.
3. **Copy Constructor:** Used to initialize an object using values of another pre-existing object of the same class.
4. **Dynamic Constructor:** A constructor that uses the `new` operator to allocate dynamic heap memory for members during object creation.

2. Deep Copying and Shallow Copying

Copy operations in C++ can lead to critical memory bugs if pointers are involved.

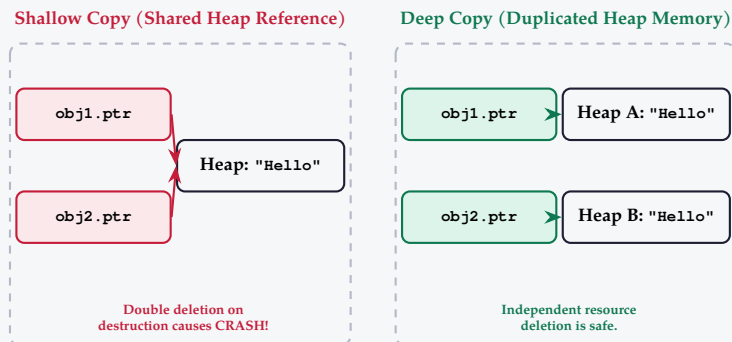
2.1 Shallow Copy vs. Deep Copy

- **Shallow Copy:** The compiler copies member variables bit-by-bit. If a class contains pointers to heap memory, only the address is copied. As a result, both the original and copied objects point to the same memory block.

Shallow Copy Risk: When either object is destroyed, its destructor runs and frees the shared heap memory. The remaining object is left with a dangling pointer, and attempting to delete it results in a double-free crash.

- **Deep Copy:** A user-defined copy constructor allocates a separate block of heap memory and copies the actual values from the source buffer. This isolates the memory blocks.

Memory Block Mappings: Shallow Copy vs. Deep Copy



Code Example: String copy constructor (Deep Copy)

```
#include <iostream>
#include <cstring>
class String {
private:
    char *str;
    int len;
public:
    String(const char *s) {
        len = strlen(s);
        str = new char[len + 1];
        strcpy(str, s);
    }
    // Deep Copy Constructor
    String(const String &source) {
        len = source.len;
        str = new char[len + 1]; // Allocate separate heap memory
        strcpy(str, source.str); // Copy actual data contents
        std::cout << "Copy Constructor Called (Deep Copy)" << std::endl;
    }
    ~String() {
        delete[] str; // Free local pointer safely
    }
    void display() { std::cout << str << std::endl; }
};
```

3. Constructor Overloading & Destructors

C++ allows multiple constructors in a class, selecting the correct one based on arguments. It also ensures proper cleanup when objects go out of scope.

3.1 Constructor Overloading and Default Arguments

Classes can have multiple constructors with different parameter signatures. They can also contain default arguments.

Constructor 1 (Default): `Complex() { real = imag = 0; }`

Constructor 2 (Parameterized): `Complex(double r, double i = 0) { real = r; imag = i; }`

3.2 Destructors

Destructor: A special member function of a class that is called automatically when an object goes out of scope or is explicitly deleted. Its purpose is to release resource allocations (like heap memory, open file handlers, socket connections).

Key Destructor Rules:

- **Naming:** Same name as class, prefixed with a tilde symbol (~).
- **Arguments:** Cannot take any parameters or return values. Overloading is not possible.
- **Execution Order:** Objects are destroyed in the **reverse order** of their creation (Last In, First Out).

Code Example: Tracing Constructor and Destructor Executions

```
#include <iostream>
class Trace {
private:
    int id;
public:
    Trace(int val) : id(val) {
        std::cout << "Constructor called for " << id << std::endl;
    }
    ~Trace() {
        std::cout << "Destructor called for " << id << std::endl;
    }
};

void runLocal() {
    std::cout << "--- Enter local scope ---" << std::endl;
    Trace localObj(2);
    std::cout << "--- Exit local scope ---" << std::endl;
}

int main() {
    std::cout << "--- Main Start ---" << std::endl;
    Trace globalObj(1);
    runLocal();
    std::cout << "--- Main End ---" << std::endl;
    return 0;
}
```

The code above outputs:

```
--- Main Start ---  
Constructor called for 1  
--- Enter local scope ---  
Constructor called for 2  
--- Exit local scope ---  
Destructor called for 2  
--- Main End ---  
Destructor called for 1
```

Quick Revision — Module IV

Constructors and Destructors | OE-EC604C

Key Constructor Syntax

Default Constructor	<code>MyClass() { x = 0; }</code>
Parameterized	<code>MyClass(int val) : x(val) { } // uses initializer list</code>
Copy Constructor	<code>MyClass(const MyClass &src) { x = src.x; } // reference is mandatory</code>
Destructor	<code>~MyClass() { /* release heap memory / close files */ }</code>

One-Line Definitions

- **Constructor:** A special member function with the class name that initializes objects upon instantiation.
- **Default Constructor:** A constructor that takes no parameters (either empty or with all arguments defaulted).
- **Copy Constructor:** A constructor used to create an object as a clone of an existing object of the same class.
- **Dynamic Constructor:** A constructor that allocates heap memory dynamically during object initialization.
- **Destructor:** A special member function prefixed with ~ that deallocates resources when an object is destroyed.
- **Shallow Copy:** A member-by-member copy of an object, copying pointer addresses rather than the target data.
- **Deep Copy:** A copy operation that duplicates the heap resources pointed to by an object, creating an independent clone.
- **Initializer List:** A comma-separated list prefixing a constructor body used to initialize data members directly.
- **Double Deletion:** A crash state occurring when two objects point to the same heap block and both attempt to delete it.

Frequently Asked / Viva Questions

Q1. Why is it mandatory to pass the argument by reference in a copy constructor?

If passed by value, the function call itself would invoke the copy constructor to clone the parameter, causing infinite recursion until the stack overflows. E.g., `MyClass(MyClass source)` is incorrect.

Frequently Asked / Viva Questions (contd.)**Q2. Can a class have more than one destructor?**

No. Destructors cannot accept arguments, which means they cannot be overloaded. Every class has exactly one destructor.

Q3. Under what conditions does the C++ compiler write a default copy constructor for you?

If a class does not define any copy constructor, the compiler automatically generates a default copy constructor, which performs a shallow copy.

Q4. Explain the order of execution of constructors and destructors in hybrid structures.

Constructors are invoked in the order of inheritance (base classes first, then derived classes). Destructors run in the exact reverse order.

Q5. Is it possible to declare a private constructor in C++?

Yes. A private constructor prevents objects from being created outside the class scope. This pattern is commonly used in the Singleton Design Pattern.

Q6. What is the difference between copy constructor and assignment operator (=)?

The copy constructor initializes a newly created object using a source object (e.g., `MyClass a = b;`). The assignment operator copies values to an already existing object (e.g., `a = b;`).

Q7. Does a destructor release the memory of the object itself?

No. A destructor only frees auxiliary resources allocated *inside* the object (e.g., heap pointers, file descriptors). The memory for the object structure itself is freed by the stack or heap manager.

Q8. Why can't we declare a constructor as `virtual`?

Virtual functions rely on the V-Table (virtual table) pointer inside an object. During construction, the object is not fully formed, and the V-Ptr is not yet initialized. Thus, constructors cannot be virtual.

Q9. What is the purpose of declaring copy constructor arguments as `const`?

Declaring the argument as `const` (e.g., `const MyClass &obj`) prevents the copy constructor from accidentally modifying the source object.

Q10. What is a member initializer list and when is it mandatory?

It is a constructor mechanism used to initialize members directly. It is mandatory for: (1) initializing reference variables, (2) initializing `const` members, and (3) calling parameterized base class constructors.

Q11. Can we call a constructor explicitly like a normal function?

Yes, we can call it explicitly (e.g., `Complex c = Complex(1.0, 2.0);`), which creates a temporary object that is then copied or moved.

Frequently Asked / Viva Questions (contd.)

Q12. What happens if an exception is thrown inside a constructor?

If an exception is thrown in a constructor, the object is considered only partially constructed, and its destructor will not run. This can lead to memory leaks unless resources are managed via smart pointers.

Comparison: Shallow Copy vs. Deep Copy

Feature	Shallow Copy	Deep Copy
Heap Memory	Shares the existing heap memory between objects.	Allocates new, independent memory blocks on the heap.
Pointer Safety	Pointer address is copied. Vulnerable to dangling pointers.	Pointers point to separate blocks. Safe from cross-corruption.
Crash Risk	High. Double deletion in destructors crashes the program.	Safe. Destructors free distinct blocks independently.
Default Behavior	Executed by default compiler copy constructors.	Must be manually coded by overriding the copy constructor.
Execution Speed	Extremely fast (simple bitwise copy).	Slightly slower (calls heap allocator and copies bytes).